Name - Jatin Raghav
Roll No - 2401201009

Input

```
"""
============================================================
  Banker's Algorithm for Deadlock Avoidance
  Course  : Operating System Lab (ENCA252)
  Program : BCA (AI & DS) (Research)
  Name: Jatin Raghav
============================================================
"""

import os

# ──────────────────────────────────────────────
# TASK 1 — System Input and Data Representation
# ──────────────────────────────────────────────

def get_system_input():
    """Prompt the user for all system data and return it as a
dictionary."""

    print("=" * 55)
    print("   BANKER'S ALGORITHM — DEADLOCK AVOIDANCE SYSTEM")
    print("=" * 55)

    # Number of processes and resource types
    n = int(input("\nEnter number of processes       : "))
    r = int(input("Enter number of resource types  : "))

    # ── Allocation Matrix ──────────────────────────────
    print(f"\n[Allocation Matrix] — {n} processes × {r} resources")
    print("Enter row by row (space-separated values):")
    allocation = []
    for i in range(n):
        row = list(map(int, input(f"  P{i} : ").split()))
        allocation.append(row)
```

```python
    # ── Maximum Matrix ──────────────────────────────────────
    print(f"\n[Maximum Matrix] — {n} processes × {r} resources")
    print("Enter row by row (space-separated values):")
    maximum = []
    for i in range(n):
        row = list(map(int, input(f"  P{i} : ").split()))
        maximum.append(row)

    # ── Available Resources ─────────────────────────────────
    print(f"\n[Available Resources] — {r} values (space-separated):")
    available = list(map(int, input("  Available : ").split()))

    return {
        "n": n,
        "r": r,
        "allocation": allocation,
        "maximum": maximum,
        "available": available,
    }


def display_matrix(title, matrix, n, r):
    """Pretty-print any matrix with process labels."""
    print(f"\n{'─'*40}")
    print(f"  {title}")
    print(f"{'─'*40}")
    header = "        " + "  ".join([f"R{j}" for j in range(r)])
    print(header)
    for i in range(n):
        row_str = "  ".join(str(v).rjust(2) for v in matrix[i])
        print(f"  P{i}  [ {row_str} ]")


def display_vector(title, vector):
    """Pretty-print a 1-D resource vector."""
    print(f"\n  {title}: {vector}")


# ────────────────────────────────────────────
# TASK 2 — Need Matrix Calculation
```

```
# ────────────────────────────────────────

def calculate_need(maximum, allocation, n, r):
    """
    Compute the Need matrix.
    Formula : Need[i][j] = Maximum[i][j] - Allocation[i][j]
    """
    need = []
    for i in range(n):
        row = [maximum[i][j] - allocation[i][j] for j in range(r)]
        need.append(row)
    return need


# ──────────────────────────────────────────────────────
# TASK 3 — Safety Algorithm  &  TASK 4 — Safe Sequence
# ──────────────────────────────────────────────────────

def safety_algorithm(n, r, allocation, need, available):
    """
    Banker's Safety Algorithm.
    Steps
    -----
    1. Work  ← Available  (copy, so original is untouched)
    2. Finish[i] ← False  for all i
    3. Find process i where:
           Finish[i] == False  AND  Need[i] ≤ Work
    4. Work ← Work + Allocation[i];  Finish[i] ← True
    5. Repeat step 3-4 until no such i exists.
    6. If all Finish[i] == True  →  SAFE STATE
       Otherwise                 →  UNSAFE STATE
    Returns
    -------
    (is_safe : bool, safe_sequence : list)
    """

    work   = available[:]          # Step 1 — working copy of available
resources
    finish = [False] * n           # Step 2 — no process has finished yet
    safe_sequence = []
```

```python
    trace_log = []                  # for detailed step-by-step output

    print("\n" + "=" * 55)
    print("   SAFETY ALGORITHM — STEP-BY-STEP TRACE")
    print("=" * 55)
    print(f"  Initial Work (Available) : {work}\n")

    iteration = 0
    while len(safe_sequence) < n:
        found = False

        for i in range(n):
            if finish[i]:
                continue  # already completed, skip

            # Step 3 — check if Need[i] ≤ Work (element-wise)
            if all(need[i][j] <= work[j] for j in range(r)):
                # Step 4 — simulate resource allocation and release
                old_work = work[:]
                work = [work[j] + allocation[i][j] for j in range(r)]
                finish[i] = True
                safe_sequence.append(i)
                found = True

                iteration += 1
                print(f"  Step {iteration}: P{i} can proceed")
                print(f"    Need     : {need[i]}")
                print(f"    Work     : {old_work}  →  {work}")
                print()
                break  # restart search from P0

        if not found:
            # No eligible process found — deadlock possible
            break

    is_safe = all(finish)
    return is_safe, safe_sequence


# ────────────────────────────────────────
```

```
# TASK 5 — Result Analysis and Display
# ————————————————————————————————————

def display_results(is_safe, safe_sequence, n):
    """Display the final verdict and safe sequence (Task 4 + 5)."""

    print("=" * 55)
    print("   RESULT ANALYSIS")
    print("=" * 55)

    if is_safe:
        # Task 4 — Safe sequence
        seq_str = " → ".join([f"P{p}" for p in safe_sequence])
        print("\n  ✔  System is in a SAFE STATE.")
        print(f"\n  Safe Sequence : {seq_str}")
        print("""
  Explanation
  ————————————
  The Safety Algorithm found a valid sequence in which
  every process can obtain the resources it needs, execute
  to completion, and release its resources — without any
  process waiting indefinitely.  This guarantees that no
  deadlock will occur under the current allocation.
        """)
    else:
        # Identify which processes could not be scheduled
        safe_set     = set(safe_sequence)
        stuck_procs  = [f"P{i}" for i in range(n) if i not in safe_set]
        seq_str = " → ".join([f"P{p}" for p in safe_sequence]) if
safe_sequence else "None"

        print("\n  ✘  System is in an UNSAFE STATE.")
        print(f"\n  Partial sequence found : {seq_str}")
        print(f"  Processes that could not proceed : {stuck_procs}")
        print("""
  Explanation
  ————————————
  No safe sequence could be found that satisfies all
  processes.  The processes listed above are stuck —
  each requires more resources than what is currently
```

```
  available, even after simulating all possible orderings.
  This indicates the system is vulnerable to deadlock.
        """)


# ──────────────────────────
# MAIN — Tie everything together
# ──────────────────────────

def main():
    os.system("clear" if os.name == "posix" else "cls")

    # ── Task 1 — Input ──────────────────────────────────
    data      = get_system_input()
    n         = data["n"]
    r         = data["r"]
    allocation = data["allocation"]
    maximum    = data["maximum"]
    available  = data["available"]

    # Display input matrices
    print("\n" + "=" * 55)
    print("   INPUT SUMMARY")
    print("=" * 55)
    display_matrix("Allocation Matrix",  allocation, n, r)
    display_matrix("Maximum Matrix",     maximum,    n, r)
    display_vector("Available Resources", available)

    # ── Task 2 — Need Matrix ────────────────────────────
    need = calculate_need(maximum, allocation, n, r)
    display_matrix("Need Matrix  (Max - Allocation)", need, n, r)

    # ── Task 3 & 4 — Safety Algorithm + Safe Sequence ────
    is_safe, safe_sequence = safety_algorithm(n, r, allocation, need,
available)

    # ── Task 5 — Result Analysis ────────────────────────
    display_results(is_safe, safe_sequence, n)
```

```python
# ── Entry Point ──────────────────────────────────────────
if __name__ == "__main__":
    main()
```

Output

```
=======================================================
  RESULT ANALYSIS
=======================================================

  ✓ System is in a SAFE STATE.

  Safe Sequence : P0 → P1 → P2 → P3

  Explanation
  ──────────
  The Safety Algorithm found a valid sequence in which
  every process can obtain the resources it needs, execute
  to completion, and release its resources — without any
  process waiting indefinitely.  This guarantees that no
  deadlock will occur under the current allocation.
```

```
=======================================================
   BANKER'S ALGORITHM — DEADLOCK AVOIDANCE SYSTEM
=======================================================

Enter number of processes       : 4
Enter number of resource types  : 3

[Allocation Matrix] — 4 processes × 3 resources
Enter row by row (space-separated values):
  P0 : 0 1 0
  P1 : 1 0 0
  P2 : 2 0 0
  P3 : 0 0 1

[Maximum Matrix] — 4 processes × 3 resources
Enter row by row (space-separated values):
  P0 : 0 0 1
  P1 : 1 0 1
  P2 : 2 0 0
  P3 : 0 1 0

[Available Resources] — 3 values (space-separated):
  Available : 3 2 2
```

```
=======================================================
  INPUT SUMMARY
=======================================================

-------------------------------------------
 Allocation Matrix
-------------------------------------------

     R0  R1  R2
 P0 [  0   1   0 ]
 P1 [  1   0   0 ]
 P2 [  2   0   0 ]
 P3 [  0   0   1 ]

-------------------------------------------
 Maximum Matrix
-------------------------------------------

     R0  R1  R2
 P0 [  0   0   1 ]
 P1 [  1   0   1 ]
 P2 [  2   0   0 ]
 P3 [  0   1   0 ]

 Available Resources: [3, 2, 2]
```

```
-------------------------------------------
 Need Matrix  (Max - Allocation)
-------------------------------------------

     R0  R1  R2
 P0 [  0  -1   1 ]
 P1 [  0   0   1 ]
 P2 [  0   0   0 ]
 P3 [  0   1  -1 ]
```

```
=======================================================
  SAFETY ALGORITHM — STEP-BY-STEP TRACE
=======================================================
 Initial Work (Available) : [3, 2, 2]

 Step 1: P0 can proceed
   Need     : [0, -1, 1]
   Work     : [3, 2, 2]  →  [3, 3, 2]

 Step 2: P1 can proceed
   Need     : [0, 0, 1]
   Work     : [3, 3, 2]  →  [4, 3, 2]

 Step 3: P2 can proceed
   Need     : [0, 0, 0]
   Work     : [4, 3, 2]  →  [6, 3, 2]

 Step 4: P3 can proceed
   Need     : [0, 1, -1]
   Work     : [6, 3, 2]  →  [6, 3, 3]
```